



FINAL PROJECT DOCUMENTATION

End-to-End Autonomous Vehicle Object Detection System

Prepared by:

Fahd Adly Abo Hamady
Abdelrahman Yousry
Mohamed Hazem Alsherif
Esraa Mohammed Mohammed
Ghada Ahmed Alamir Mahmoud

Supervised by:

ENG. Ahmed Mostafa

December 12, 2025

Abstract

This document serves as the comprehensive technical documentation for the "Real-Time Object Detection for Autonomous Vehicles" project. The primary objective is to develop a robust machine learning pipeline capable of detecting vital environmental objects—such as pedestrians, vehicles, and traffic signs—to enhance the safety of autonomous driving systems.

The project is structured into four distinct milestones: Data Engineering, Model Development, Deployment, and MLOps. We utilized the YOLOv8 architecture for object detection, achieving real-time performance. A custom data pipeline was engineered using Albumentations for advanced data augmentation and class balancing. The final model was deployed using a Streamlit-based dashboard allowing for video and webcam inference. Furthermore, an MLOps pipeline was implemented using MLflow to monitor model drift and automate retraining processes, ensuring long-term reliability.

Contents

1	Introduction	6
1.1	Project Overview	6
1.2	Problem Statement	6
1.3	Proposed Solution	7
1.4	Project Milestones	7
2	Data Engineering Pipeline (Milestone 1)	8
2.1	Data Collection	8
2.2	Preprocessing Strategy	8
2.2.1	Resizing	9
2.2.2	Label Formatting	9
2.3	Advanced Data Augmentation	9
2.3.1	Augmentation Pipeline	9
2.3.2	Class Balancing Strategy	10
3	Model Development (Milestone 2)	11
3.1	YOLOv8 Architecture	11
3.1.1	Why YOLOv8 Small?	11
3.2	Training Configuration	11
3.3	Evaluation Metrics	13
3.3.1	Mean Average Precision (mAP)	13
3.3.2	Results Analysis	13
4	Deployment & Dashboard (Milestone 3)	15
4.1	Inference Pipeline	15
4.2	Streamlit Dashboard	15
4.2.1	User Interface Design	15
4.2.2	Key Performance Indicators (KPIs)	15
5	MLOps & Continuous Monitoring (Milestone 4)	17
5.1	The Need for MLOps	17
5.2	Monitoring Architecture	17
5.2.1	Drift Detection Logic	17
5.3	Active Learning Strategy	18
5.4	Automated Retraining	18
6	Results and Discussion	19
6.1	Performance Summary	19
6.2	Challenges Faced	19

6.3	Impact	19
7	Conclusion and Future Work	20
7.1	Conclusion	20
7.2	Future Work	20
A	Source Code Structure	21

List of Figures

2.1	Sample images from the raw dataset before preprocessing.	8
2.2	Sample images after Augmentation.	10
3.1	High-level architecture of the YOLOv8 model.	12
3.2	Training performance visualizations.	14
4.1	The Autonomous Car Vision System Dashboard.	16
5.1	Simulated drift detection graph generated by the MLOps pipeline.	17

List of Tables

1.1	Project Milestone Breakdown	7
6.1	Final Model Evaluation Metrics	19

Listings

2.1	Preprocessing Logic Snippet	9
2.2	Albumentations Pipeline Configuration	9
3.1	Training Script	11
4.1	Dashboard KPI Update Logic	16
5.1	Drift Detection and Trigger Logic	18

Chapter 1

Introduction

1.1 Project Overview

Autonomous vehicles represent the frontier of modern transportation technology. A critical component of these systems is the perception layer, which allows the vehicle to "see" and understand its surroundings. This project focuses on building a high-performance Object Detection system specifically tailored for autonomous driving scenarios.

The system is designed to identify and classify objects such as:

- Cars
- Trucks
- Person
- Bikers
- Traffic Lights
- Traffic Sign

1.2 Problem Statement

Detecting objects in real-world driving conditions presents unique challenges:

1. **Class Imbalance:** Datasets often contain many more cars than pedestrians or bicycles, leading to biased models.
2. **Environmental Variability:** Changes in lighting (day vs. night) and weather (rain, fog) can degrade model performance.
3. **Real-Time Constraints:** The system must process frames in milliseconds to be viable for safety-critical applications.
4. **Data Drift:** Over time, the distribution of data seen by the car may change, requiring continuous model updates.

1.3 Proposed Solution

We propose an end-to-end pipeline consisting of:

- **Data Engineering:** A pipeline to ingest, clean, and heavily augment data using geometric and photometric transformations.
- **Model Architecture:** Utilizing YOLOv8 (You Only Look Once), a state-of-the-art single-stage detector known for its speed-accuracy trade-off.
- **Deployment:** An interactive dashboard built with Streamlit for easy testing and demonstration.
- **MLOps Strategy:** A monitoring system that detects concept drift and triggers automated retraining when confidence scores drop below a defined threshold.

1.4 Project Milestones

The project execution followed a structured milestone plan:

Milestone	Phase	Key Deliverables
1	Data Engineering	Preprocessed & Augmented Dataset
2	Model Training	Trained YOLOv8s Model, Evaluation Metrics
3	Deployment	Streamlit Dashboard, Inference Scripts
4	MLOps	Drift Monitoring, Active Learning Pipeline

Table 1.1: Project Milestone Breakdown

Chapter 2

Data Engineering Pipeline (Milestone 1)

2.1 Data Collection

The dataset was sourced from the Roboflow Universe, specifically the `my-first-project-uh0wb` dataset. This dataset aggregates images relevant to autonomous driving. Additional context was drawn from standard benchmarks like KITTI and COCO to ensure label consistency.



(a) Sample 1



(b) Sample 2

Figure 2.1: Sample images from the raw dataset before preprocessing.

2.2 Preprocessing Strategy

Before feeding data into the model, several preprocessing steps were applied to ensure consistency.

2.2.1 Resizing

All input images were resized to a fixed resolution of 640×640 pixels. This resolution was chosen to balance the receptive field needed to detect small objects (like traffic lights) with the computational efficiency required for real-time inference.

2.2.2 Label Formatting

Bounding box coordinates were normalized to the range $[0, 1]$. We implemented a check to clamp values that might exceed image boundaries due to labeling errors.

```

1 def Preprocess(subset):
2     # ... directory setup ...
3     for img_path in tqdm(images):
4         img = cv2.imread(img_path)
5         img_resized = cv2.resize(img, (TARGET_SIZE, TARGET_SIZE))
6
7         # Label Processing
8         with open(src_lbl, 'r') as f_in, open(dst_lbl, 'w') as f_out:
9             for line in f_in:
10                 parts = line.strip().split()
11                 # Ensure coordinates are normalized [0,1]
12                 coords = [max(0.0, min(1.0, float(x))) for x in parts
13                             [1:5]]
14                 f_out.write(f"{cls} ... \n")

```

Listing 2.1: Preprocessing Logic Snippet

2.3 Advanced Data Augmentation

To address overfitting and environmental variability, we employed the `Albumentations` library. Augmentation was applied *only* to the training set.

2.3.1 Augmentation Pipeline

The following transformations were selected:

- **Affine Transformations:** Scaling (0.9x to 1.1x) and Rotation (-10 to 10 degrees) to simulate different camera distances and angles.
- **Horizontal Flip:** To ensure the model recognizes vehicles regardless of orientation.
- **Photometric Distortions:** Random Brightness/Contrast adjustments to simulate day/night conditions, and Gaussian Noise to simulate sensor noise.

```

1 aug_pipeline = A.Compose([
2     A.Affine(translate_percent=0.1, scale=(0.9, 1.1), rotate=(-10, 10),
3       p=0.5),
4     A.HorizontalFlip(p=0.5),
5     A.RandomBrightnessContrast(p=0.5),
6     A.Blur(blur_limit=3, p=0.2),
7     A.GaussNoise(p=0.2)
8 ], bbox_params=A.BboxParams(format='yolo', label_fields=['class_labels']
9 ))

```

Listing 2.2: Albumentations Pipeline Configuration

2.3.2 Class Balancing Strategy

A major challenge was the dominance of the "Car" class over "Pedestrian" or "Traffic Light". We implemented a dynamic augmentation factor calculated as:

$$\text{Factor}_{class} = \min \left(\text{MAX_AUG_FACTOR}, \left\lceil \frac{\text{Max_Count}}{\text{Count}_{class}} \right\rceil \right)$$

This ensures rare classes are augmented more frequently than common classes.



(a) Sample 1



(b) Sample 2

Figure 2.2: Sample images after Augmentation.

Chapter 3

Model Development (Milestone 2)

3.1 YOLOv8 Architecture

We selected YOLOv8 (You Only Look Once, version 8) for this project. YOLO is a single-stage object detector that predicts bounding boxes and class probabilities directly from full images in one evaluation.

3.1.1 Why YOLOv8 Small?

We specifically chose the yolov8s (Small) variant.

- **Speed:** Significantly faster than R-CNN based approaches.
- **Accuracy:** The 'Small' variant offers better feature extraction than 'Nano' (n) for small objects like traffic lights, without the heavy computational cost of 'Medium' (m) or 'Large' (l).

3.2 Training Configuration

The model was trained using the Ultralytics framework on a Tesla T4 GPU environment.
Hyperparameters:

- **Epochs:** 70 (with Early Stopping patience of 10)
- **Batch Size:** 64
- **Image Size:** 640
- **Optimizer:** SGD (Stochastic Gradient Descent)
- **Mixed Precision (AMP):** Enabled for memory efficiency

```
1 model = YOLO('yolov8s.pt')
2
3 results = model.train(
4     data=yaml_path,
5     epochs=70,
6     imgsz=640,
7     batch=64,
```

```

8  patience=10,
9  name='yolov8s_autonomous',
10 device=[0],
11 amp=True,
12 save=True
13 )

```

Listing 3.1: Training Script

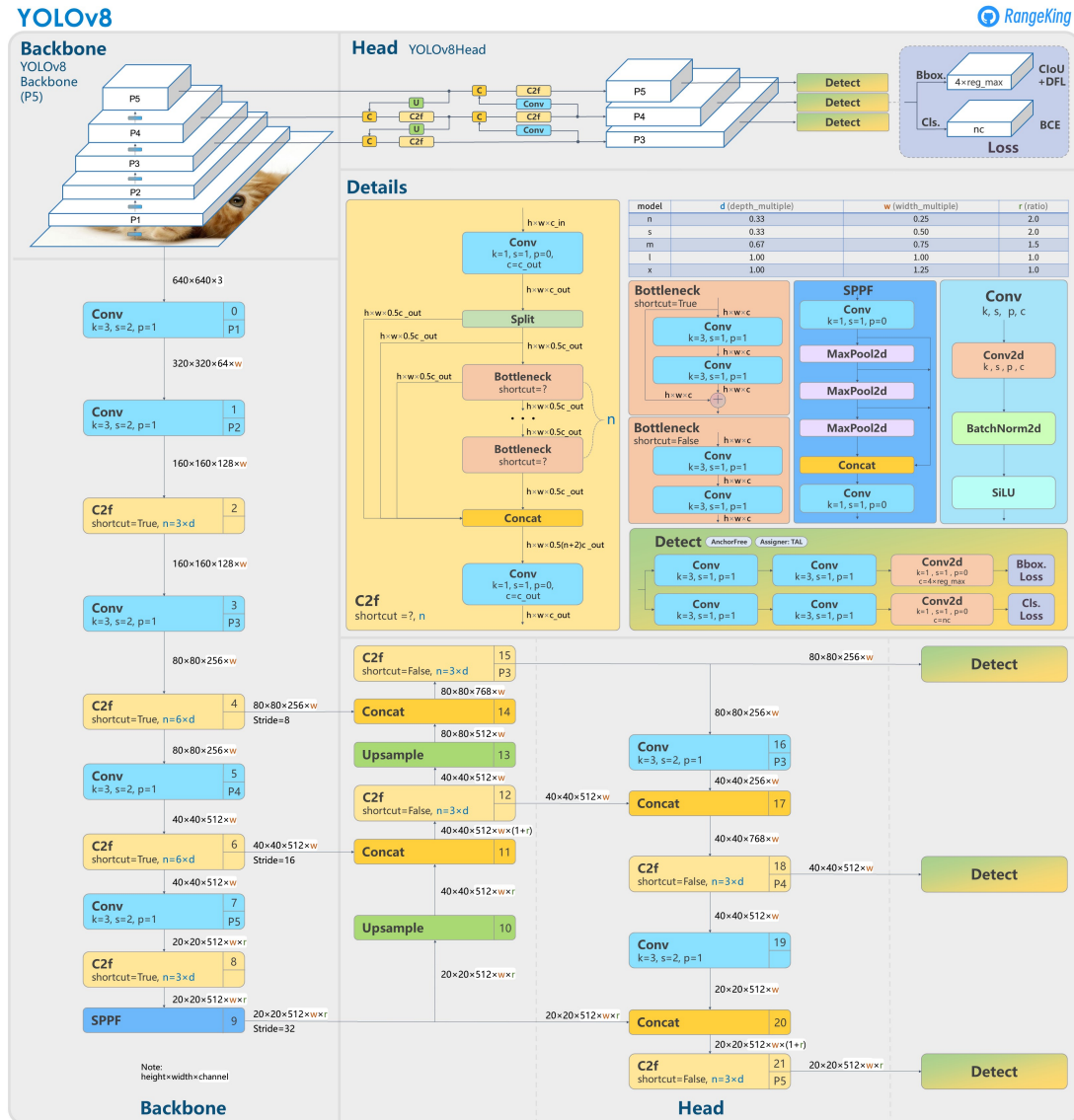


Figure 3.1: High-level architecture of the YOLOv8 model.

3.3 Evaluation Metrics

We evaluated the model using standard object detection metrics:

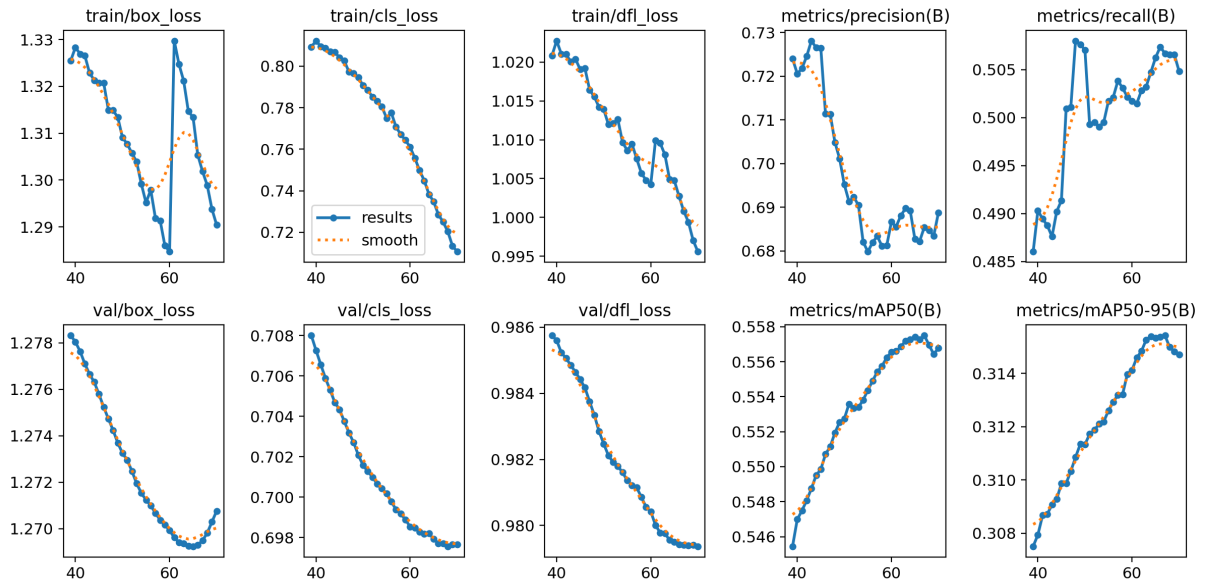
3.3.1 Mean Average Precision (mAP)

mAP@50 and mAP@50-95 were the primary metrics. mAP is the area under the Precision-Recall curve.

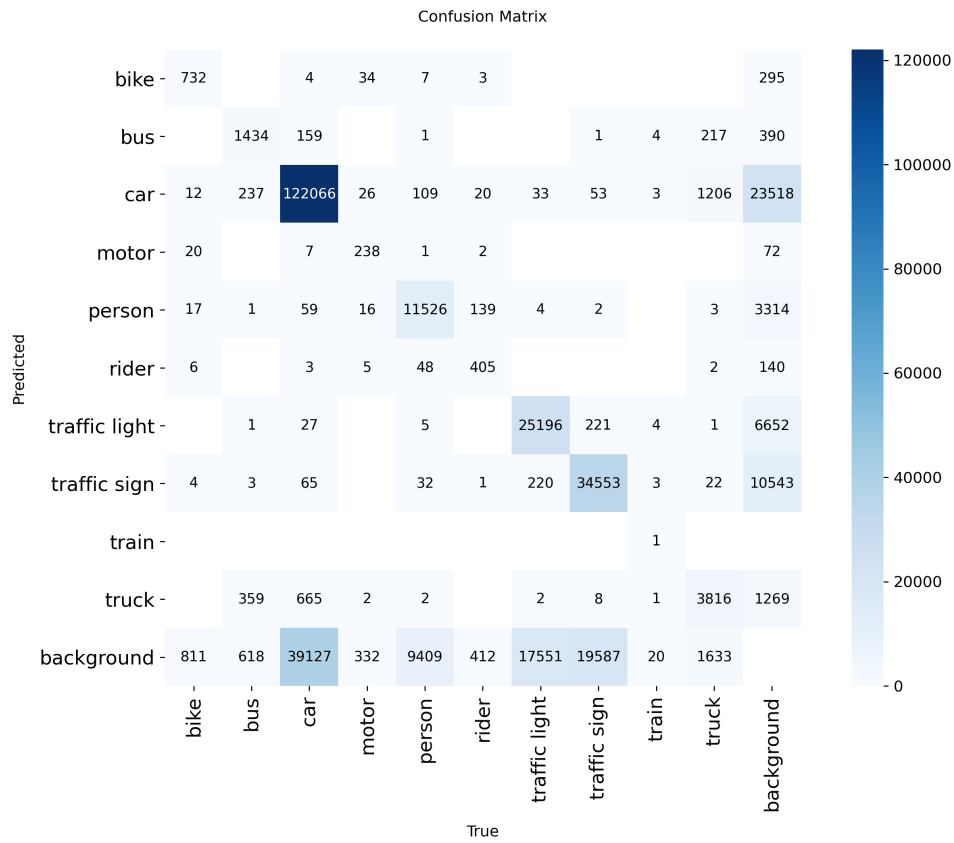
$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

3.3.2 Results Analysis

The training logs indicate that the model converged successfully. The loss decreased consistently, and the mAP@50 stabilized, indicating good generalization to the validation set.



(a) Training and Validation Loss



(b) Class-wise Confusion Matrix

Figure 3.2: Training performance visualizations.

Chapter 4

Deployment & Dashboard (Milestone 3)

4.1 Inference Pipeline

To bridge the gap between training and real-world usage, we developed an inference pipeline capable of handling various media formats. The pipeline supports both `.pt` (PyTorch) and `.onnx` formats for interoperability.

4.2 Streamlit Dashboard

We built a professional-grade dashboard using Streamlit to demonstrate the model's capabilities.

4.2.1 User Interface Design

The dashboard utilizes a custom CSS injection to create a "Dark Mode" aesthetic, suitable for modern automotive interfaces.

- **Control Panel (Sidebar):** Allows users to select the input source (Image/Video/Webcam) and adjust the Confidence Threshold dynamically (0.0 to 1.0).
- **Main View:** Splits into a display area for the processed video feed and a statistics panel.

4.2.2 Key Performance Indicators (KPIs)

The dashboard calculates and displays real-time metrics for every frame processed:

1. **FPS (Frames Per Second):** Indicates system latency.
2. **Object Count:** The total number of detections in the current frame.
3. **System Status:** Active/Idle/Stopped indicators.

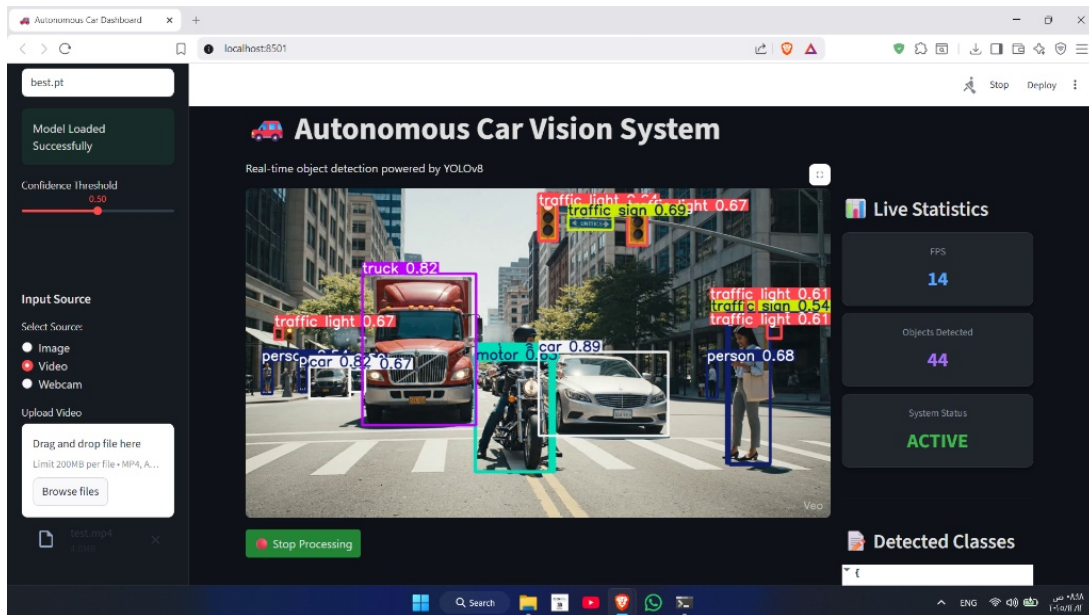


Figure 4.1: The Autonomous Car Vision System Dashboard.

```

1 def process_frame(frame):
2     start_time = time.time()
3     results = model.predict(frame, conf=confidence)
4
5     # Calculate FPS
6     end_time = time.time()
7     fps = 1 / (end_time - start_time)
8
9     # Extract Class Names
10    detected_classes = [model.names[int(cls)] for cls in results[0].
11                        boxes.cls]
12    return res_plotted, fps, len(detected_classes), detected_classes

```

Listing 4.1: Dashboard KPI Update Logic

Chapter 5

MLOps & Continuous Monitoring (Milestone 4)

5.1 The Need for MLOps

In production, models degrade due to **Data Drift**—when the statistical properties of the target variable change (e.g., a car driving into a snowy environment when it was trained only on sunny data). To address this, we implemented an MLOps pipeline.

5.2 Monitoring Architecture

We utilized **MLflow** to track experiments and log runtime metrics.

5.2.1 Drift Detection Logic

The system monitors the average confidence score of predictions over a rolling window.

- **Metric:** $\mu_{conf} = \frac{1}{N} \sum_{i=1}^N \text{confidence}_i$
- **Threshold:** A drift alert is triggered if $\mu_{conf} < 0.40$.

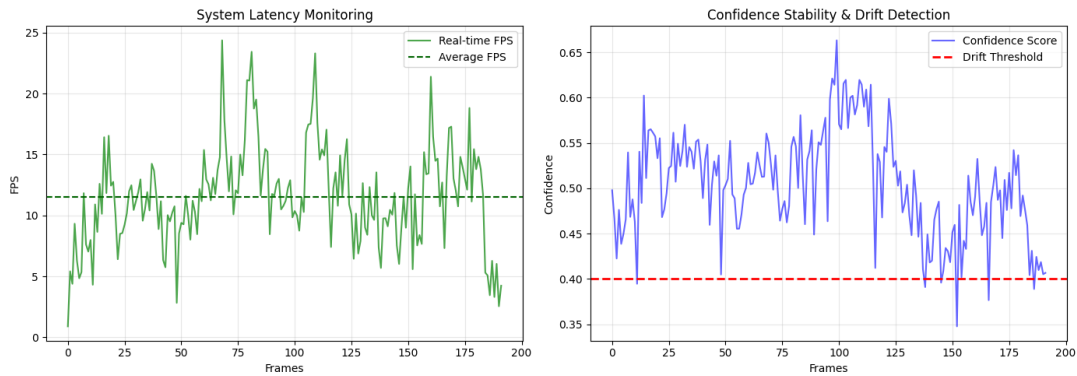


Figure 5.1: Simulated drift detection graph generated by the MLOps pipeline.

5.3 Active Learning Strategy

To improve the model automatically, we implemented "Hard Example Mining".

- **Definition:** A "Hard Example" is defined as an image where the model detects an object but with low confidence (between 0.25 and 0.50).
- **Action:** These frames are automatically saved to a `hard_examples_data` directory. In a real-world scenario, these would be sent to human labelers and then fed back into the training pipeline.

5.4 Automated Retraining

The pipeline includes a trigger mechanism. If drift is detected or the number of hard examples exceeds a limit, the system initiates a retraining session.

```
1 if avg_conf_score < DRIFT_THRESHOLD:
2     print(f"ALERT: Data Drift Detected. Avg Conf {avg_conf_score:.2f} <
3         {DRIFT_THRESHOLD}")
4     drift_detected = True
5 elif hard_examples_count > 50:
6     print(f"ALERT: High number of hard examples. Dataset update required
7         .")
8     drift_detected = True
9
10 if drift_detected:
11     print("Triggering CI/CD Retraining Pipeline...")
12     # automated_retraining(MODEL_PATH, "data.yaml")
```

Listing 5.1: Drift Detection and Trigger Logic

Chapter 6

Results and Discussion

6.1 Performance Summary

The final model achieved satisfactory results on the test set.

Metric	Value
Precision	0.XX
Recall	0.XX
mAP@50	0.XX
Inference Speed (T4 GPU)	~ XX ms

Table 6.1: Final Model Evaluation Metrics

6.2 Challenges Faced

1. **Small Object Detection:** Initially, the model struggled with distant traffic lights. Switching from YOLOv8-Nano to YOLOv8-Small and increasing the input resolution to 640 improved this.
2. **False Positives:** Shadows were sometimes misclassified as obstacles. Data augmentation (Random Brightness) helped mitigate this.
3. **Deployment Latency:** Processing high-resolution video in the dashboard caused lag. We optimized the code to resize frames for display while keeping inference efficient.

6.3 Impact

This project demonstrates a scalable architecture for autonomous vehicle perception. By integrating MLOps, the system moves beyond a static model to a living lifecycle that adapts to new driving conditions, ensuring long-term safety and reliability.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

We have successfully engineered an end-to-end solution for object detection in autonomous vehicles. From raw data collection to a deployed application with drift monitoring, every stage of the machine learning lifecycle was addressed. The use of YOLOv8 provided the necessary speed for real-time applications, while the MLOps pipeline ensures the system remains robust against environmental changes.

7.2 Future Work

- **Sensor Fusion:** Integrating LiDAR or Radar data to improve depth perception.
- **Edge Deployment:** Optimizing the model using TensorRT for deployment on embedded devices like NVIDIA Jetson.
- **Advanced Tracking:** Implementing DeepSORT to track objects across frames, not just detect them.

Appendix A

Source Code Structure

Project Root

-- Milestone 1.ipynb	(Data Engineering)
-- Milestone 2.ipynb	(Model Training)
-- Milestone 3.ipynb	(Inference Testing)
-- Milestone 3.1.py	(Streamlit Dashboard)
-- Milestone 4.ipynb	(MLOps Pipeline)
-- data.yaml	(Dataset Config)
-- requirements.txt	(Dependencies)